

```
In [1]: # =====
# 1. IMPORTS
# =====

import pandas as pd
import numpy as np
import tensorflow as tf

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    f1_score,
    recall_score
)
from sklearn.utils.class_weight import compute_class_weight

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization, Conv1D, MaxPooling1D, Flatten
from tensorflow.keras.callbacks import EarlyStopping
from tensorflow.keras.optimizers import Adam
```

```
In [2]: # =====
# 2. LOAD DATA
# =====

df = pd.read_csv("raw_table.csv")

df_model = df.dropna(subset=["ATSEG"]).copy()
```

```
In [3]: # =====
# 3. FEATURE ENGINEERING
# =====

df_model["TOTAL_PRESCRIPTIONS"] = (
    df_model["UC_TRX"] + df_model["ORAL_TRX"] + df_model["IL23_TRX"]
)

df_model["ENGAGEMENT_INDEX"] = (
    df_model["RTE"] + df_model["DETAILS"] + df_model["SAMPLES"]
)

df_model["ENGAGEMENT_RATIO"] = df_model["ENGAGEMENT_INDEX"] / (
    df_model["TOTAL_PRESCRIPTIONS"] + 1
)

df_model["IL23_RATIO"] = df_model["IL23_TRX"] / (df_model["TOTAL_PRESCRIPTIONS"] + 1)
df_model["ORAL_RATIO"] = df_model["ORAL_TRX"] / (df_model["TOTAL_PRESCRIPTIONS"] + 1)
df_model["UC_RATIO"] = df_model["UC_TRX"] / (df_model["TOTAL_PRESCRIPTIONS"] + 1)

df_model["RX_PER_ENGAGEMENT"] = df_model["TOTAL_PRESCRIPTIONS"] / (
    df_model["ENGAGEMENT_INDEX"] + 1
)
```

```
In [4]: # =====
# 4. AGGREGATE BY DOCTOR
# =====

features_to_agg = [
    "TOTAL_PRESCRIPTIONS",
    "ENGAGEMENT_INDEX",
    "ENGAGEMENT_RATIO",
    "RX_PER_ENGAGEMENT",
    "UC_TRX",
    "ORAL_TRX",
    "IL23_TRX",
    "IL23_RATIO",
    "ORAL_RATIO",
    "UC_RATIO",
    "RTE",
    "SAMPLES",
    "DETAILS",
    "COPAY",
```

```

"DIRECTMAIL",
"SPK"
]

agg_df = df_model.groupby("NUEVO_ID").agg({
    **{col: ["mean", "max", "sum", "std"] for col in features_to_agg},
    "ATSEG": "first"
})

agg_df.columns = [
    "_".join(col).strip() if col[1] else col[0]
    for col in agg_df.columns
]

agg_df = agg_df.reset_index()
agg_df = agg_df.fillna(0)

```

```

In [5]: # =====
# 5. TRAIN / TEST SPLIT
# =====

X = agg_df.drop(columns=["NUEVO_ID", "ATSEG_first"])
y = agg_df["ATSEG_first"]

le = LabelEncoder()
y_encoded = le.fit_transform(y)
y_cat = tf.keras.utils.to_categorical(y_encoded)

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y_cat,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

y_true = np.argmax(y_test, axis=1)

print(le.classes_)

['SEG_A' 'SEG_B' 'SEG_C']

```

```

In [6]: # =====
# 6. CLASS WEIGHTS
# =====

class_weights_array = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_encoded),
    y=y_encoded
)

class_weights = dict(enumerate(class_weights_array))

# Ajuste manual para dar más importancia a SEG_C
class_weights_manual = {
    0: 1.0, # SEG_A
    1: 1.3, # SEG_B
    2: 3.0 # SEG_C
}

print(class_weights)
print(class_weights_manual)

{0: np.float64(0.6191591216567801), 1: np.float64(1.1843336319299294), 2: np.float64(1.849968905472637)}
{0: 1.0, 1: 1.3, 2: 3.0}

```

```

In [7]: # =====
# 7. FUNCTION TO EVALUATE MODELS
# =====

results = []

def evaluate_model(model_name, model, X_eval, y_test):

```

```

y_pred_prob = model.predict(X_eval)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y_test, axis=1)

print("\n=====")
print(model_name)
print("=====")
print(classification_report(y_true, y_pred, target_names=le.classes_))
print(confusion_matrix(y_true, y_pred))

f1_per_class = f1_score(y_true, y_pred, average=None)
recall_per_class = recall_score(y_true, y_pred, average=None)

results.append({
    "model": model_name,
    "accuracy": float(accuracy_score(y_true, y_pred)),
    "macro_f1": float(f1_score(y_true, y_pred, average="macro")),
    "weighted_f1": float(f1_score(y_true, y_pred, average="weighted")),
    "seg_c_f1": float(f1_per_class[2]),
    "seg_c_recall": float(recall_per_class[2])
})

```

```

In [18]: # =====
# 8. MODEL 1: BASIC MLP
# =====

mlp_model = Sequential([
    Dense(128, activation="relu", input_shape=(X_train_scaled.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),

    Dense(64, activation="relu"),
    BatchNormalization(),
    Dropout(0.3),

    Dense(32, activation="relu"),
    Dropout(0.2),

    Dense(3, activation="softmax")
])

mlp_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

mlp_model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=60,
    batch_size=64,
    class_weight=class_weights_manual,
    callbacks=[early_stop],
    verbose=1
)

evaluate_model("MLP", mlp_model, X_test_scaled, y_test)

```

Epoch 1/60

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

119/119 ————— 2s 5ms/step - accuracy: 0.4961 - loss: 1.5877 - val_accuracy: 0.5788 - val_loss: 0.8925
Epoch 2/60
119/119 ————— 1s 4ms/step - accuracy: 0.5355 - loss: 1.4434 - val_accuracy: 0.5583 - val_loss: 0.8825
Epoch 3/60
119/119 ————— 0s 3ms/step - accuracy: 0.5401 - loss: 1.3909 - val_accuracy: 0.5641 - val_loss: 0.8748
Epoch 4/60
119/119 ————— 0s 4ms/step - accuracy: 0.5505 - loss: 1.3486 - val_accuracy: 0.5741 - val_loss: 0.8731
Epoch 5/60
119/119 ————— 0s 4ms/step - accuracy: 0.5619 - loss: 1.3459 - val_accuracy: 0.5840 - val_loss: 0.8802
Epoch 6/60
119/119 ————— 0s 4ms/step - accuracy: 0.5705 - loss: 1.3332 - val_accuracy: 0.5935 - val_loss: 0.8700
Epoch 7/60
119/119 ————— 0s 3ms/step - accuracy: 0.5694 - loss: 1.3114 - val_accuracy: 0.5898 - val_loss: 0.8789
Epoch 8/60
119/119 ————— 0s 3ms/step - accuracy: 0.5783 - loss: 1.3113 - val_accuracy: 0.5856 - val_loss: 0.8855
Epoch 9/60
119/119 ————— 0s 3ms/step - accuracy: 0.5747 - loss: 1.3074 - val_accuracy: 0.5930 - val_loss: 0.8765
Epoch 10/60
119/119 ————— 0s 3ms/step - accuracy: 0.5823 - loss: 1.3055 - val_accuracy: 0.5882 - val_loss: 0.8966
Epoch 11/60
119/119 ————— 1s 4ms/step - accuracy: 0.5888 - loss: 1.3007 - val_accuracy: 0.5903 - val_loss: 0.8836
Epoch 12/60
119/119 ————— 1s 7ms/step - accuracy: 0.5930 - loss: 1.3048 - val_accuracy: 0.5898 - val_loss: 0.8792
Epoch 13/60
119/119 ————— 1s 4ms/step - accuracy: 0.5902 - loss: 1.2971 - val_accuracy: 0.6050 - val_loss: 0.8693
Epoch 14/60
119/119 ————— 1s 5ms/step - accuracy: 0.5955 - loss: 1.2866 - val_accuracy: 0.5861 - val_loss: 0.8895
Epoch 15/60
119/119 ————— 1s 5ms/step - accuracy: 0.5866 - loss: 1.2907 - val_accuracy: 0.5924 - val_loss: 0.8761
Epoch 16/60
119/119 ————— 0s 4ms/step - accuracy: 0.6014 - loss: 1.2927 - val_accuracy: 0.5882 - val_loss: 0.8789
Epoch 17/60
119/119 ————— 0s 3ms/step - accuracy: 0.5944 - loss: 1.2885 - val_accuracy: 0.5877 - val_loss: 0.8782
Epoch 18/60
119/119 ————— 0s 4ms/step - accuracy: 0.5933 - loss: 1.2857 - val_accuracy: 0.5898 - val_loss: 0.8777
Epoch 19/60
119/119 ————— 1s 4ms/step - accuracy: 0.6000 - loss: 1.2848 - val_accuracy: 0.5951 - val_loss: 0.8768
Epoch 20/60
119/119 ————— 1s 4ms/step - accuracy: 0.5968 - loss: 1.2858 - val_accuracy: 0.5840 - val_loss: 0.8824
Epoch 21/60
119/119 ————— 0s 4ms/step - accuracy: 0.5899 - loss: 1.2858 - val_accuracy: 0.5861 - val_loss: 0.8843
Epoch 22/60
119/119 ————— 1s 4ms/step - accuracy: 0.5989 - loss: 1.2820 - val_accuracy: 0.5772 - val_loss: 0.8862
Epoch 23/60
119/119 ————— 1s 4ms/step - accuracy: 0.5984 - loss: 1.2792 - val_accuracy: 0.5830 - val_loss: 0.8810
Epoch 24/60
119/119 ————— 1s 4ms/step - accuracy: 0.5997 - loss: 1.2722 - val_accuracy: 0.5940 - val_loss: 0.8740
Epoch 25/60
119/119 ————— 0s 3ms/step - accuracy: 0.5938 - loss: 1.2747 - val_accuracy: 0.5882 - val_loss: 0.8782
Epoch 26/60
119/119 ————— 0s 3ms/step - accuracy: 0.6008 - loss: 1.2734 - val_accuracy: 0.5830 - val_loss: 0.8761

Epoch 27/60
119/119 ————— 0s 3ms/step - accuracy: 0.5925 - loss: 1.2814 - val_accuracy: 0.5798 - val_loss: 0.8838
Epoch 28/60
119/119 ————— 1s 5ms/step - accuracy: 0.5958 - loss: 1.2706 - val_accuracy: 0.5851 - val_loss: 0.8692
Epoch 29/60
119/119 ————— 0s 3ms/step - accuracy: 0.6007 - loss: 1.2622 - val_accuracy: 0.5998 - val_loss: 0.8852
Epoch 30/60
119/119 ————— 0s 3ms/step - accuracy: 0.5997 - loss: 1.2721 - val_accuracy: 0.5961 - val_loss: 0.8771
Epoch 31/60
119/119 ————— 0s 4ms/step - accuracy: 0.6093 - loss: 1.2612 - val_accuracy: 0.5819 - val_loss: 0.8808
Epoch 32/60
119/119 ————— 0s 4ms/step - accuracy: 0.6004 - loss: 1.2638 - val_accuracy: 0.5767 - val_loss: 0.8844
Epoch 33/60
119/119 ————— 0s 4ms/step - accuracy: 0.6030 - loss: 1.2674 - val_accuracy: 0.5977 - val_loss: 0.8767
Epoch 34/60
119/119 ————— 1s 6ms/step - accuracy: 0.5995 - loss: 1.2690 - val_accuracy: 0.5872 - val_loss: 0.8762
Epoch 35/60
119/119 ————— 1s 5ms/step - accuracy: 0.5997 - loss: 1.2666 - val_accuracy: 0.5835 - val_loss: 0.8693
Epoch 36/60
119/119 ————— 0s 4ms/step - accuracy: 0.6045 - loss: 1.2615 - val_accuracy: 0.6008 - val_loss: 0.8669
Epoch 37/60
119/119 ————— 0s 4ms/step - accuracy: 0.6045 - loss: 1.2574 - val_accuracy: 0.5741 - val_loss: 0.8845
Epoch 38/60
119/119 ————— 0s 4ms/step - accuracy: 0.5988 - loss: 1.2658 - val_accuracy: 0.5935 - val_loss: 0.8712
Epoch 39/60
119/119 ————— 1s 4ms/step - accuracy: 0.6080 - loss: 1.2598 - val_accuracy: 0.5783 - val_loss: 0.8771
Epoch 40/60
119/119 ————— 0s 4ms/step - accuracy: 0.6028 - loss: 1.2601 - val_accuracy: 0.5825 - val_loss: 0.8806
Epoch 41/60
119/119 ————— 1s 4ms/step - accuracy: 0.6024 - loss: 1.2502 - val_accuracy: 0.5809 - val_loss: 0.8802
Epoch 42/60
119/119 ————— 0s 3ms/step - accuracy: 0.5989 - loss: 1.2615 - val_accuracy: 0.5856 - val_loss: 0.8790
Epoch 43/60
119/119 ————— 0s 4ms/step - accuracy: 0.6051 - loss: 1.2581 - val_accuracy: 0.5809 - val_loss: 0.8736
Epoch 44/60
119/119 ————— 0s 4ms/step - accuracy: 0.6028 - loss: 1.2644 - val_accuracy: 0.5798 - val_loss: 0.8708
Epoch 45/60
119/119 ————— 0s 3ms/step - accuracy: 0.6063 - loss: 1.2524 - val_accuracy: 0.5809 - val_loss: 0.8903
Epoch 46/60
119/119 ————— 1s 4ms/step - accuracy: 0.6021 - loss: 1.2567 - val_accuracy: 0.5783 - val_loss: 0.8838
Epoch 47/60
119/119 ————— 0s 4ms/step - accuracy: 0.6059 - loss: 1.2531 - val_accuracy: 0.6056 - val_loss: 0.8611
Epoch 48/60
119/119 ————— 0s 3ms/step - accuracy: 0.6049 - loss: 1.2527 - val_accuracy: 0.5909 - val_loss: 0.8763
Epoch 49/60
119/119 ————— 0s 3ms/step - accuracy: 0.6130 - loss: 1.2585 - val_accuracy: 0.5788 - val_loss: 0.8770
Epoch 50/60
119/119 ————— 0s 3ms/step - accuracy: 0.6098 - loss: 1.2471 - val_accuracy: 0.5846 - val_loss: 0.8685
Epoch 51/60
119/119 ————— 0s 4ms/step - accuracy: 0.6068 - loss: 1.2528 - val_accuracy: 0.5966 - val_loss: 0.8702
Epoch 52/60
119/119 ————— 0s 4ms/step - accuracy: 0.6190 - loss: 1.2512 - val_accuracy: 0.5825 - val_loss: 0.8900

```

7
Epoch 53/60
119/119 ————— 0s 4ms/step - accuracy: 0.6053 - loss: 1.2507 - val_accuracy: 0.5877 - val_loss: 0.879
4
Epoch 54/60
119/119 ————— 0s 4ms/step - accuracy: 0.6063 - loss: 1.2416 - val_accuracy: 0.5861 - val_loss: 0.871
8
Epoch 55/60
119/119 ————— 0s 4ms/step - accuracy: 0.6129 - loss: 1.2485 - val_accuracy: 0.5935 - val_loss: 0.879
8
Epoch 56/60
119/119 ————— 0s 4ms/step - accuracy: 0.6102 - loss: 1.2437 - val_accuracy: 0.5924 - val_loss: 0.864
6
Epoch 57/60
119/119 ————— 1s 7ms/step - accuracy: 0.6152 - loss: 1.2379 - val_accuracy: 0.5840 - val_loss: 0.884
2
Epoch 58/60
119/119 ————— 1s 6ms/step - accuracy: 0.6097 - loss: 1.2358 - val_accuracy: 0.6024 - val_loss: 0.859
9
Epoch 59/60
119/119 ————— 0s 4ms/step - accuracy: 0.6214 - loss: 1.2359 - val_accuracy: 0.5751 - val_loss: 0.894
2
Epoch 60/60
119/119 ————— 1s 4ms/step - accuracy: 0.6108 - loss: 1.2429 - val_accuracy: 0.5856 - val_loss: 0.885
3
75/75 ————— 0s 2ms/step

```

```

=====
MLP
=====

```

	precision	recall	f1-score	support
SEG_A	0.75	0.78	0.76	1281
SEG_B	0.56	0.29	0.38	670
SEG_C	0.36	0.59	0.45	429
accuracy			0.61	2380
macro avg	0.56	0.55	0.53	2380
weighted avg	0.63	0.61	0.60	2380

```

[[996  86 199]
 [224 195 251]
 [110  66 253]]

```

```

In [19]: # =====
# 9. PREPARE DATA FOR CNN
# =====

X_train_cnn = X_train_scaled.reshape(X_train_scaled.shape[0], X_train_scaled.shape[1], 1)
X_test_cnn = X_test_scaled.reshape(X_test_scaled.shape[0], X_test_scaled.shape[1], 1)

```

```

In [20]: # =====
# 10. MODEL 2: CNN 1D
# =====

cnn_model = Sequential([
    Conv1D(64, kernel_size=5, activation="relu", input_shape=(X_train_cnn.shape[1], 1)),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),
    Dropout(0.3),

    Conv1D(32, kernel_size=3, activation="relu"),
    BatchNormalization(),
    Dropout(0.3),

    Flatten(),

    Dense(64, activation="relu"),
    Dropout(0.3),

    Dense(3, activation="softmax")
])

cnn_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

```

```
)  
  
cnn_model.fit(  
    X_train_cnn,  
    y_train,  
    validation_split=0.2,  
    epochs=60,  
    batch_size=64,  
    class_weight=class_weights_manual,  
    callbacks=[early_stop],  
    verbose=1  
)  
  
evaluate_model("CNN", cnn_model, X_test_cnn, y_test)
```

Epoch 1/60

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass a  
n `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` objec  
t as the first layer in the model instead.  
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

119/119 ————— 4s 18ms/step - accuracy: 0.5299 - loss: 1.6183 - val_accuracy: 0.3330 - val_loss: 1.0608
Epoch 2/60
119/119 ————— 2s 17ms/step - accuracy: 0.5511 - loss: 1.3956 - val_accuracy: 0.4853 - val_loss: 0.9953
Epoch 3/60
119/119 ————— 3s 22ms/step - accuracy: 0.5736 - loss: 1.3426 - val_accuracy: 0.4422 - val_loss: 0.9929
Epoch 4/60
119/119 ————— 2s 15ms/step - accuracy: 0.5773 - loss: 1.3197 - val_accuracy: 0.5762 - val_loss: 0.9019
Epoch 5/60
119/119 ————— 1s 12ms/step - accuracy: 0.5849 - loss: 1.3090 - val_accuracy: 0.5441 - val_loss: 0.8996
Epoch 6/60
119/119 ————— 3s 13ms/step - accuracy: 0.5802 - loss: 1.3012 - val_accuracy: 0.5614 - val_loss: 0.8942
Epoch 7/60
119/119 ————— 2s 15ms/step - accuracy: 0.5904 - loss: 1.3029 - val_accuracy: 0.5788 - val_loss: 0.8764
Epoch 8/60
119/119 ————— 2s 13ms/step - accuracy: 0.5812 - loss: 1.3005 - val_accuracy: 0.5877 - val_loss: 0.8750
Epoch 9/60
119/119 ————— 2s 19ms/step - accuracy: 0.5919 - loss: 1.2871 - val_accuracy: 0.5909 - val_loss: 0.8625
Epoch 10/60
119/119 ————— 2s 14ms/step - accuracy: 0.5911 - loss: 1.2840 - val_accuracy: 0.5919 - val_loss: 0.8658
Epoch 11/60
119/119 ————— 2s 15ms/step - accuracy: 0.5899 - loss: 1.2791 - val_accuracy: 0.5914 - val_loss: 0.8543
Epoch 12/60
119/119 ————— 2s 16ms/step - accuracy: 0.5991 - loss: 1.2726 - val_accuracy: 0.5872 - val_loss: 0.8904
Epoch 13/60
119/119 ————— 2s 17ms/step - accuracy: 0.5965 - loss: 1.2770 - val_accuracy: 0.5814 - val_loss: 0.8791
Epoch 14/60
119/119 ————— 2s 13ms/step - accuracy: 0.5972 - loss: 1.2669 - val_accuracy: 0.5762 - val_loss: 0.8971
Epoch 15/60
119/119 ————— 2s 16ms/step - accuracy: 0.6030 - loss: 1.2702 - val_accuracy: 0.5909 - val_loss: 0.8576
Epoch 16/60
119/119 ————— 2s 15ms/step - accuracy: 0.6021 - loss: 1.2554 - val_accuracy: 0.5830 - val_loss: 0.8745
Epoch 17/60
119/119 ————— 2s 13ms/step - accuracy: 0.5992 - loss: 1.2551 - val_accuracy: 0.5830 - val_loss: 0.8870
Epoch 18/60
119/119 ————— 2s 16ms/step - accuracy: 0.6053 - loss: 1.2570 - val_accuracy: 0.5940 - val_loss: 0.8590
Epoch 19/60
119/119 ————— 2s 17ms/step - accuracy: 0.6080 - loss: 1.2525 - val_accuracy: 0.5835 - val_loss: 0.8712
Epoch 20/60
119/119 ————— 2s 19ms/step - accuracy: 0.6101 - loss: 1.2522 - val_accuracy: 0.5861 - val_loss: 0.8671
Epoch 21/60
119/119 ————— 3s 22ms/step - accuracy: 0.6089 - loss: 1.2422 - val_accuracy: 0.5877 - val_loss: 0.8751
Epoch 22/60
119/119 ————— 2s 15ms/step - accuracy: 0.6032 - loss: 1.2517 - val_accuracy: 0.5882 - val_loss: 0.8748
Epoch 23/60
119/119 ————— 2s 14ms/step - accuracy: 0.6213 - loss: 1.2400 - val_accuracy: 0.5825 - val_loss: 0.8922
Epoch 24/60
119/119 ————— 2s 14ms/step - accuracy: 0.6126 - loss: 1.2408 - val_accuracy: 0.5783 - val_loss: 0.8928
Epoch 25/60
119/119 ————— 2s 13ms/step - accuracy: 0.6091 - loss: 1.2379 - val_accuracy: 0.5814 - val_loss: 0.8806
Epoch 26/60
119/119 ————— 2s 13ms/step - accuracy: 0.6084 - loss: 1.2354 - val_accuracy: 0.5830 - val_loss: 0.8718

=====

CNN

=====

	precision	recall	f1-score	support
SEG_A	0.76	0.75	0.76	1281
SEG_B	0.55	0.32	0.40	670
SEG_C	0.34	0.59	0.43	429
accuracy			0.60	2380
macro avg	0.55	0.55	0.53	2380
weighted avg	0.63	0.60	0.60	2380

```
[[959  97 225]
 [195 212 263]
 [101  76 252]]
```

```
In [21]: # =====
# 11. MODEL 3: CNN WITH FOCAL LOSS
# =====

def focal_loss(gamma=2.0, alpha=[1.0, 1.2, 2.5]):
    alpha = tf.constant(alpha, dtype=tf.float32)

    def loss(y_true, y_pred):
        y_pred = tf.clip_by_value(y_pred, 1e-7, 1.0 - 1e-7)
        cross_entropy = -y_true * tf.math.log(y_pred)
        weight = alpha * tf.pow(1 - y_pred, gamma)
        loss = weight * cross_entropy
        return tf.reduce_sum(loss, axis=1)

    return loss
```

```
In [22]: cnn_focal_model = Sequential([
    Conv1D(64, kernel_size=5, activation="relu", input_shape=(X_train_cnn.shape[1], 1)),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),
    Dropout(0.3),

    Conv1D(32, kernel_size=3, activation="relu"),
    BatchNormalization(),
    Dropout(0.3),

    Flatten(),

    Dense(64, activation="relu"),
    Dropout(0.3),

    Dense(3, activation="softmax")
])

cnn_focal_model.compile(
    optimizer=Adam(learning_rate=0.001),
    loss=focal_loss(gamma=2.0, alpha=[1.0, 1.3, 3.0]),
    metrics=["accuracy"]
)

cnn_focal_model.fit(
    X_train_cnn,
    y_train,
    validation_split=0.2,
    epochs=60,
    batch_size=64,
    callbacks=[early_stop],
    verbose=1
)

evaluate_model("CNN_Focal_Loss", cnn_focal_model, X_test_cnn, y_test)
```

Epoch 1/60

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass a
n `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` objec
t as the first layer in the model instead.
```

```
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

119/119  4s 15ms/step - accuracy: 0.4835 - loss: 1.0354 - val_accuracy: 0.2421 - val_loss: 0.6727
Epoch 2/60
119/119  2s 13ms/step - accuracy: 0.5144 - loss: 0.6603 - val_accuracy: 0.3109 - val_loss: 0.6212
Epoch 3/60
119/119  2s 20ms/step - accuracy: 0.5505 - loss: 0.5987 - val_accuracy: 0.4338 - val_loss: 0.5919
Epoch 4/60
119/119  2s 14ms/step - accuracy: 0.5644 - loss: 0.5743 - val_accuracy: 0.5168 - val_loss: 0.5634
Epoch 5/60
119/119  2s 13ms/step - accuracy: 0.5641 - loss: 0.5584 - val_accuracy: 0.6035 - val_loss: 0.5662
Epoch 6/60
119/119  2s 15ms/step - accuracy: 0.5702 - loss: 0.5567 - val_accuracy: 0.5762 - val_loss: 0.5502
Epoch 7/60
119/119  2s 14ms/step - accuracy: 0.5884 - loss: 0.5466 - val_accuracy: 0.5693 - val_loss: 0.5441
Epoch 8/60
119/119  2s 14ms/step - accuracy: 0.5760 - loss: 0.5511 - val_accuracy: 0.5636 - val_loss: 0.5519
Epoch 9/60
119/119  3s 20ms/step - accuracy: 0.5821 - loss: 0.5442 - val_accuracy: 0.5678 - val_loss: 0.5425
Epoch 10/60
119/119  2s 14ms/step - accuracy: 0.5891 - loss: 0.5399 - val_accuracy: 0.5867 - val_loss: 0.5453
Epoch 11/60
119/119  2s 13ms/step - accuracy: 0.5921 - loss: 0.5397 - val_accuracy: 0.5699 - val_loss: 0.5457
Epoch 12/60
119/119  2s 13ms/step - accuracy: 0.5929 - loss: 0.5387 - val_accuracy: 0.5814 - val_loss: 0.5421
Epoch 13/60
119/119  1s 12ms/step - accuracy: 0.5937 - loss: 0.5350 - val_accuracy: 0.5809 - val_loss: 0.5504
Epoch 14/60
119/119  2s 14ms/step - accuracy: 0.5890 - loss: 0.5373 - val_accuracy: 0.5725 - val_loss: 0.5366
Epoch 15/60
119/119  2s 15ms/step - accuracy: 0.5878 - loss: 0.5354 - val_accuracy: 0.5777 - val_loss: 0.5373
Epoch 16/60
119/119  3s 21ms/step - accuracy: 0.6068 - loss: 0.5264 - val_accuracy: 0.5940 - val_loss: 0.5415
Epoch 17/60
119/119  2s 16ms/step - accuracy: 0.5858 - loss: 0.5303 - val_accuracy: 0.6056 - val_loss: 0.5415
Epoch 18/60
119/119  2s 12ms/step - accuracy: 0.5924 - loss: 0.5329 - val_accuracy: 0.5783 - val_loss: 0.5422
Epoch 19/60
119/119  2s 14ms/step - accuracy: 0.5871 - loss: 0.5306 - val_accuracy: 0.6066 - val_loss: 0.5412
Epoch 20/60
119/119  2s 16ms/step - accuracy: 0.6070 - loss: 0.5279 - val_accuracy: 0.5783 - val_loss: 0.5379
Epoch 21/60
119/119  2s 14ms/step - accuracy: 0.5993 - loss: 0.5235 - val_accuracy: 0.5825 - val_loss: 0.5408
Epoch 22/60
119/119  3s 22ms/step - accuracy: 0.5976 - loss: 0.5236 - val_accuracy: 0.5835 - val_loss: 0.5393
Epoch 23/60
119/119  2s 16ms/step - accuracy: 0.6020 - loss: 0.5219 - val_accuracy: 0.5872 - val_loss: 0.5468
Epoch 24/60
119/119  2s 13ms/step - accuracy: 0.6063 - loss: 0.5231 - val_accuracy: 0.5940 - val_loss: 0.5430
Epoch 25/60
119/119  1s 12ms/step - accuracy: 0.5944 - loss: 0.5238 - val_accuracy: 0.5914 - val_loss: 0.5494
Epoch 26/60
119/119  2s 14ms/step - accuracy: 0.6008 - loss: 0.5191 - val_accuracy: 0.6071 - val_loss: 0.5437

Epoch 27/60
119/119 ————— 2s 15ms/step - accuracy: 0.5992 - loss: 0.5168 - val_accuracy: 0.5893 - val_loss: 0.5458
Epoch 28/60
119/119 ————— 2s 15ms/step - accuracy: 0.5993 - loss: 0.5187 - val_accuracy: 0.6192 - val_loss: 0.5519
Epoch 29/60
119/119 ————— 2s 18ms/step - accuracy: 0.6039 - loss: 0.5172 - val_accuracy: 0.5846 - val_loss: 0.5453
75/75 ————— 0s 3ms/step

=====

CNN_Focal_Loss

=====

	precision	recall	f1-score	support
SEG_A	0.77	0.73	0.75	1281
SEG_B	0.54	0.20	0.29	670
SEG_C	0.32	0.69	0.44	429
accuracy			0.57	2380
macro avg	0.54	0.54	0.49	2380
weighted avg	0.62	0.57	0.56	2380

[[932 84 265]
[182 135 353]
[103 31 295]]

```
In [23]: # =====  
# 12. FINAL COMPARISON TABLE  
# =====  
  
results_df = pd.DataFrame(results).round(3)  
  
results_df = results_df.sort_values(  
    by=["seg_c_f1", "macro_f1"],  
    ascending=False  
)  
  
results_df
```

Out[23]:

	model	accuracy	macro_f1	weighted_f1	seg_c_f1	seg_c_recall
4	MLP	0.607	0.531	0.599	0.447	0.590
6	CNN_Focal_Loss	0.572	0.493	0.563	0.440	0.688
5	CNN	0.598	0.530	0.598	0.431	0.587
2	CNN_Focal_Loss	0.571	0.491	0.563	0.429	0.676
0	MLP	0.596	0.519	0.591	0.428	0.592
3	CNN_Focal_Loss	0.303	0.252	0.269	0.340	0.960
1	CNN	0.192	0.118	0.080	0.309	1.000

```
In [24]: # =====  
# 13. OPTIONAL: SAVE RESULTS  
# =====  
  
results_df.to_csv("neural_network_results.csv", index=False)
```

Modelo final

```
In [25]: # =====  
# 1. REINICIAR RESULTADOS  
# =====  
results = []
```

```
In [26]: # =====  
# 2. DEFINIR MODELO MLP  
# =====  
  
mlp_model = Sequential([  
    Dense(128, activation="relu", input_shape=(X_train_scaled.shape[1],)),
```

```

        BatchNormalization(),
        Dropout(0.3),

        Dense(64, activation="relu"),
        BatchNormalization(),
        Dropout(0.3),

        Dense(32, activation="relu"),
        Dropout(0.2),

        Dense(3, activation="softmax")
    ])

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

In [27]: # =====
# 3. COMPILAR MODELO
# =====

mlp_model.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

```

```

In [28]: # =====
# 4. EARLY STOPPING
# =====

early_stop = EarlyStopping(
    monitor="val_loss",
    patience=15,
    restore_best_weights=True
)

```

```

In [29]: # =====
# 5. ENTRENAMIENTO
# =====

history = mlp_model.fit(
    X_train_scaled,
    y_train,
    validation_split=0.2,
    epochs=80,
    batch_size=64,
    class_weight={0:1.0, 1:1.3, 2:2.3}, # ayuda a SEG_C sin exagerar
    callbacks=[early_stop],
    verbose=1
)

```

Epoch 1/80
119/119 ————— 6s 10ms/step - accuracy: 0.4968 - loss: 1.6299 - val_accuracy: 0.5814 - val_loss: 0.9058
Epoch 2/80
119/119 ————— 1s 4ms/step - accuracy: 0.5463 - loss: 1.4072 - val_accuracy: 0.5809 - val_loss: 0.8967
Epoch 3/80
119/119 ————— 0s 3ms/step - accuracy: 0.5598 - loss: 1.3317 - val_accuracy: 0.5882 - val_loss: 0.8738
Epoch 4/80
119/119 ————— 0s 3ms/step - accuracy: 0.5740 - loss: 1.3016 - val_accuracy: 0.5935 - val_loss: 0.8693
Epoch 5/80
119/119 ————— 0s 4ms/step - accuracy: 0.5815 - loss: 1.2683 - val_accuracy: 0.5840 - val_loss: 0.8726
Epoch 6/80
119/119 ————— 1s 4ms/step - accuracy: 0.5791 - loss: 1.2600 - val_accuracy: 0.5898 - val_loss: 0.8595
Epoch 7/80
119/119 ————— 0s 3ms/step - accuracy: 0.5833 - loss: 1.2518 - val_accuracy: 0.5856 - val_loss: 0.8678
Epoch 8/80
119/119 ————— 0s 3ms/step - accuracy: 0.5887 - loss: 1.2358 - val_accuracy: 0.5924 - val_loss: 0.8699
Epoch 9/80
119/119 ————— 0s 3ms/step - accuracy: 0.5955 - loss: 1.2161 - val_accuracy: 0.5987 - val_loss: 0.8614
Epoch 10/80
119/119 ————— 0s 4ms/step - accuracy: 0.5951 - loss: 1.2127 - val_accuracy: 0.6024 - val_loss: 0.8621
Epoch 11/80
119/119 ————— 1s 4ms/step - accuracy: 0.5970 - loss: 1.2076 - val_accuracy: 0.6113 - val_loss: 0.8497
Epoch 12/80
119/119 ————— 0s 4ms/step - accuracy: 0.6100 - loss: 1.2040 - val_accuracy: 0.6050 - val_loss: 0.8555
Epoch 13/80
119/119 ————— 1s 5ms/step - accuracy: 0.6113 - loss: 1.1931 - val_accuracy: 0.6061 - val_loss: 0.8602
Epoch 14/80
119/119 ————— 1s 5ms/step - accuracy: 0.6117 - loss: 1.1947 - val_accuracy: 0.6218 - val_loss: 0.8544
Epoch 15/80
119/119 ————— 1s 6ms/step - accuracy: 0.6108 - loss: 1.1965 - val_accuracy: 0.6129 - val_loss: 0.8572
Epoch 16/80
119/119 ————— 0s 3ms/step - accuracy: 0.6146 - loss: 1.1905 - val_accuracy: 0.6145 - val_loss: 0.8626
Epoch 17/80
119/119 ————— 0s 3ms/step - accuracy: 0.6151 - loss: 1.1899 - val_accuracy: 0.6087 - val_loss: 0.8534
Epoch 18/80
119/119 ————— 0s 4ms/step - accuracy: 0.6202 - loss: 1.1826 - val_accuracy: 0.6098 - val_loss: 0.8522
Epoch 19/80
119/119 ————— 1s 4ms/step - accuracy: 0.6211 - loss: 1.1827 - val_accuracy: 0.6134 - val_loss: 0.8528
Epoch 20/80
119/119 ————— 1s 4ms/step - accuracy: 0.6285 - loss: 1.1778 - val_accuracy: 0.6197 - val_loss: 0.8512
Epoch 21/80
119/119 ————— 0s 4ms/step - accuracy: 0.6177 - loss: 1.1795 - val_accuracy: 0.6129 - val_loss: 0.8469
Epoch 22/80
119/119 ————— 0s 4ms/step - accuracy: 0.6151 - loss: 1.1774 - val_accuracy: 0.6087 - val_loss: 0.8495
Epoch 23/80
119/119 ————— 1s 4ms/step - accuracy: 0.6255 - loss: 1.1753 - val_accuracy: 0.6098 - val_loss: 0.8507
Epoch 24/80
119/119 ————— 1s 4ms/step - accuracy: 0.6257 - loss: 1.1693 - val_accuracy: 0.6082 - val_loss: 0.8525
Epoch 25/80
119/119 ————— 1s 5ms/step - accuracy: 0.6225 - loss: 1.1764 - val_accuracy: 0.6045 - val_loss: 0.8554
Epoch 26/80
119/119 ————— 0s 3ms/step - accuracy: 0.6263 - loss: 1.1742 - val_accuracy: 0.6035 - val_loss: 0.858

7
Epoch 27/80
119/119  0s 3ms/step - accuracy: 0.6206 - loss: 1.1740 - val_accuracy: 0.6092 - val_loss: 0.8493
Epoch 28/80
119/119  1s 4ms/step - accuracy: 0.6280 - loss: 1.1635 - val_accuracy: 0.6061 - val_loss: 0.8545
Epoch 29/80
119/119  1s 4ms/step - accuracy: 0.6238 - loss: 1.1687 - val_accuracy: 0.6113 - val_loss: 0.8524
Epoch 30/80
119/119  0s 3ms/step - accuracy: 0.6193 - loss: 1.1735 - val_accuracy: 0.6166 - val_loss: 0.8469
Epoch 31/80
119/119  0s 3ms/step - accuracy: 0.6223 - loss: 1.1614 - val_accuracy: 0.6129 - val_loss: 0.8474
Epoch 32/80
119/119  0s 3ms/step - accuracy: 0.6177 - loss: 1.1601 - val_accuracy: 0.6187 - val_loss: 0.8442
Epoch 33/80
119/119  0s 3ms/step - accuracy: 0.6295 - loss: 1.1595 - val_accuracy: 0.6150 - val_loss: 0.8492
Epoch 34/80
119/119  0s 3ms/step - accuracy: 0.6261 - loss: 1.1645 - val_accuracy: 0.6161 - val_loss: 0.8491
Epoch 35/80
119/119  0s 3ms/step - accuracy: 0.6243 - loss: 1.1543 - val_accuracy: 0.6134 - val_loss: 0.8448
Epoch 36/80
119/119  0s 3ms/step - accuracy: 0.6340 - loss: 1.1552 - val_accuracy: 0.6145 - val_loss: 0.8477
Epoch 37/80
119/119  0s 3ms/step - accuracy: 0.6278 - loss: 1.1533 - val_accuracy: 0.6103 - val_loss: 0.8429
Epoch 38/80
119/119  1s 6ms/step - accuracy: 0.6225 - loss: 1.1607 - val_accuracy: 0.6119 - val_loss: 0.8511
Epoch 39/80
119/119  1s 6ms/step - accuracy: 0.6251 - loss: 1.1538 - val_accuracy: 0.6171 - val_loss: 0.8455
Epoch 40/80
119/119  1s 4ms/step - accuracy: 0.6343 - loss: 1.1557 - val_accuracy: 0.6155 - val_loss: 0.8519
Epoch 41/80
119/119  1s 4ms/step - accuracy: 0.6335 - loss: 1.1490 - val_accuracy: 0.6134 - val_loss: 0.8504
Epoch 42/80
119/119  0s 3ms/step - accuracy: 0.6247 - loss: 1.1572 - val_accuracy: 0.6166 - val_loss: 0.8487
Epoch 43/80
119/119  0s 3ms/step - accuracy: 0.6243 - loss: 1.1549 - val_accuracy: 0.6113 - val_loss: 0.8501
Epoch 44/80
119/119  0s 4ms/step - accuracy: 0.6320 - loss: 1.1532 - val_accuracy: 0.6124 - val_loss: 0.8536
Epoch 45/80
119/119  1s 5ms/step - accuracy: 0.6282 - loss: 1.1522 - val_accuracy: 0.6171 - val_loss: 0.8407
Epoch 46/80
119/119  1s 5ms/step - accuracy: 0.6297 - loss: 1.1514 - val_accuracy: 0.6113 - val_loss: 0.8523
Epoch 47/80
119/119  1s 5ms/step - accuracy: 0.6236 - loss: 1.1540 - val_accuracy: 0.6155 - val_loss: 0.8510
Epoch 48/80
119/119  1s 5ms/step - accuracy: 0.6265 - loss: 1.1524 - val_accuracy: 0.6224 - val_loss: 0.8400
Epoch 49/80
119/119  1s 4ms/step - accuracy: 0.6328 - loss: 1.1468 - val_accuracy: 0.6171 - val_loss: 0.8440
Epoch 50/80
119/119  0s 3ms/step - accuracy: 0.6324 - loss: 1.1457 - val_accuracy: 0.6176 - val_loss: 0.8445
Epoch 51/80
119/119  0s 3ms/step - accuracy: 0.6315 - loss: 1.1432 - val_accuracy: 0.6113 - val_loss: 0.8491
Epoch 52/80

119/119 ————— 0s 3ms/step - accuracy: 0.6268 - loss: 1.1452 - val_accuracy: 0.6171 - val_loss: 0.8458
Epoch 53/80
119/119 ————— 0s 3ms/step - accuracy: 0.6330 - loss: 1.1392 - val_accuracy: 0.6245 - val_loss: 0.8483
Epoch 54/80
119/119 ————— 0s 3ms/step - accuracy: 0.6372 - loss: 1.1403 - val_accuracy: 0.6155 - val_loss: 0.8546
Epoch 55/80
119/119 ————— 0s 3ms/step - accuracy: 0.6324 - loss: 1.1387 - val_accuracy: 0.6182 - val_loss: 0.8474
Epoch 56/80
119/119 ————— 0s 3ms/step - accuracy: 0.6347 - loss: 1.1364 - val_accuracy: 0.6213 - val_loss: 0.8513
Epoch 57/80
119/119 ————— 0s 3ms/step - accuracy: 0.6309 - loss: 1.1349 - val_accuracy: 0.6213 - val_loss: 0.8434
Epoch 58/80
119/119 ————— 1s 4ms/step - accuracy: 0.6285 - loss: 1.1450 - val_accuracy: 0.6208 - val_loss: 0.8371
Epoch 59/80
119/119 ————— 1s 4ms/step - accuracy: 0.6408 - loss: 1.1336 - val_accuracy: 0.6203 - val_loss: 0.8428
Epoch 60/80
119/119 ————— 0s 4ms/step - accuracy: 0.6406 - loss: 1.1291 - val_accuracy: 0.6203 - val_loss: 0.8503
Epoch 61/80
119/119 ————— 1s 6ms/step - accuracy: 0.6362 - loss: 1.1417 - val_accuracy: 0.6187 - val_loss: 0.8510
Epoch 62/80
119/119 ————— 1s 8ms/step - accuracy: 0.6357 - loss: 1.1354 - val_accuracy: 0.6239 - val_loss: 0.8414
Epoch 63/80
119/119 ————— 1s 5ms/step - accuracy: 0.6378 - loss: 1.1338 - val_accuracy: 0.6124 - val_loss: 0.8398
Epoch 64/80
119/119 ————— 0s 4ms/step - accuracy: 0.6365 - loss: 1.1342 - val_accuracy: 0.6161 - val_loss: 0.8491
Epoch 65/80
119/119 ————— 0s 3ms/step - accuracy: 0.6397 - loss: 1.1321 - val_accuracy: 0.6182 - val_loss: 0.8458
Epoch 66/80
119/119 ————— 0s 3ms/step - accuracy: 0.6343 - loss: 1.1361 - val_accuracy: 0.6234 - val_loss: 0.8435
Epoch 67/80
119/119 ————— 1s 5ms/step - accuracy: 0.6394 - loss: 1.1314 - val_accuracy: 0.6192 - val_loss: 0.8490
Epoch 68/80
119/119 ————— 1s 5ms/step - accuracy: 0.6357 - loss: 1.1299 - val_accuracy: 0.6166 - val_loss: 0.8450
Epoch 69/80
119/119 ————— 1s 4ms/step - accuracy: 0.6337 - loss: 1.1312 - val_accuracy: 0.6250 - val_loss: 0.8364
Epoch 70/80
119/119 ————— 0s 3ms/step - accuracy: 0.6415 - loss: 1.1251 - val_accuracy: 0.6239 - val_loss: 0.8492
Epoch 71/80
119/119 ————— 0s 3ms/step - accuracy: 0.6372 - loss: 1.1283 - val_accuracy: 0.6271 - val_loss: 0.8374
Epoch 72/80
119/119 ————— 0s 3ms/step - accuracy: 0.6374 - loss: 1.1305 - val_accuracy: 0.6255 - val_loss: 0.8406
Epoch 73/80
119/119 ————— 1s 4ms/step - accuracy: 0.6370 - loss: 1.1317 - val_accuracy: 0.6197 - val_loss: 0.8448
Epoch 74/80
119/119 ————— 1s 4ms/step - accuracy: 0.6390 - loss: 1.1259 - val_accuracy: 0.6171 - val_loss: 0.8428
Epoch 75/80
119/119 ————— 0s 3ms/step - accuracy: 0.6420 - loss: 1.1256 - val_accuracy: 0.6161 - val_loss: 0.8576
Epoch 76/80
119/119 ————— 0s 3ms/step - accuracy: 0.6423 - loss: 1.1289 - val_accuracy: 0.6187 - val_loss: 0.8488
Epoch 77/80
119/119 ————— 1s 4ms/step - accuracy: 0.6298 - loss: 1.1282 - val_accuracy: 0.6261 - val_loss: 0.8476

Epoch 78/80
119/119 ————— **0s** 4ms/step - accuracy: 0.6465 - loss: 1.1225 - val_accuracy: 0.6155 - val_loss: 0.8464
Epoch 79/80
119/119 ————— **0s** 4ms/step - accuracy: 0.6408 - loss: 1.1233 - val_accuracy: 0.6187 - val_loss: 0.8480
Epoch 80/80
119/119 ————— **1s** 4ms/step - accuracy: 0.6450 - loss: 1.1197 - val_accuracy: 0.6203 - val_loss: 0.8498

```
In [30]: # =====
# 6. EVALUACIÓN
# =====

y_pred_prob = mlp_model.predict(X_test_scaled)
y_pred = y_pred_prob.argmax(axis=1)
y_true = y_test.argmax(axis=1)

print(classification_report(y_true, y_pred))
print(confusion_matrix(y_true, y_pred))
```

75/75 ————— **0s** 3ms/step

	precision	recall	f1-score	support
0	0.74	0.79	0.77	1281
1	0.56	0.41	0.47	670
2	0.39	0.47	0.43	429
accuracy			0.63	2380
macro avg	0.56	0.56	0.55	2380
weighted avg	0.63	0.63	0.62	2380

```
[[1018 117 146]
 [ 229 272 169]
 [ 130  97 202]]
```

```
In [31]: # =====
# 7. GUARDAR RESULTADOS
# =====

f1_per_class = f1_score(y_true, y_pred, average=None)
recall_per_class = recall_score(y_true, y_pred, average=None)

results.append({
    "model": "MLP_FINAL",
    "accuracy": float(accuracy_score(y_true, y_pred)),
    "macro_f1": float(f1_score(y_true, y_pred, average="macro")),
    "weighted_f1": float(f1_score(y_true, y_pred, average="weighted")),
    "seg_c_f1": float(f1_per_class[2]),
    "seg_c_recall": float(recall_per_class[2])
})

import pandas as pd
pd.DataFrame(results).round(3)
```

```
Out[31]:
```

	model	accuracy	macro_f1	weighted_f1	seg_c_f1	seg_c_recall
0	MLP_FINAL	0.627	0.555	0.622	0.427	0.471

CNN temporal

```
In [34]: features = [
    "UC_TRX", "ORAL_TRX", "IL23_TRX",
    "RTE", "SAMPLES", "COPAY",
    "DIRECTMAIL", "SPK", "DETAILS"
]

df_time = df.dropna(subset=["ATSEG"]).copy()

X_time = []
y_time = []

for doctor_id in df_time["NUEVO_ID"].unique():
    temp = df_time[df_time["NUEVO_ID"] == doctor_id].sort_values("WEEK_ID")
```



```

        if len(temp) == 86:
            X_time.append(temp[features].values)
            y_time.append(temp["ATSEG"].iloc[0])

X_time = np.array(X_time)
y_time = np.array(y_time)

print(X_time.shape)
print(y_time.shape)

```

```

(11899, 86, 9)
(11899,)

```

```

In [35]: # =====
# 3. ENCODING DEL TARGET
# =====

le_time = LabelEncoder()
y_encoded = le_time.fit_transform(y_time)
y_cat = tf.keras.utils.to_categorical(y_encoded)

print(le_time.classes_)

```

```
['SEG_A' 'SEG_B' 'SEG_C']
```

```

In [36]: # =====
# 4. TRAIN / TEST SPLIT
# =====

X_train_time, X_test_time, y_train_time, y_test_time = train_test_split(
    X_time,
    y_cat,
    test_size=0.2,
    random_state=42,
    stratify=y_encoded
)

```

```

In [37]: # =====
# 5. ESCALAR VARIABLES TEMPORALES
# =====

n_train, n_weeks, n_features = X_train_time.shape
n_test = X_test_time.shape[0]

scaler = StandardScaler()

X_train_resaped = X_train_time.reshape(-1, n_features)
X_test_resaped = X_test_time.reshape(-1, n_features)

X_train_scaled = scaler.fit_transform(X_train_resaped)
X_test_scaled = scaler.transform(X_test_resaped)

X_train_time_scaled = X_train_scaled.reshape(n_train, n_weeks, n_features)
X_test_time_scaled = X_test_scaled.reshape(n_test, n_weeks, n_features)

```

```

In [38]: # =====
# 6. CLASS WEIGHTS
# =====

class_weights_array = compute_class_weight(
    class_weight="balanced",
    classes=np.unique(y_encoded),
    y=y_encoded
)

class_weights = dict(enumerate(class_weights_array))

# Puedes usar este manual si quieres ayudar más a SEG_C
class_weights_manual = {
    0: 1.0,
    1: 1.3,
    2: 2.3
}

print(class_weights)

```

```
{0: np.float64(0.6191591216567801), 1: np.float64(1.1843336319299294), 2: np.float64(1.849968905472637)}
```

```
In [39]: # =====
# 7. CNN TEMPORAL
# =====

temporal_cnn = Sequential([
    Conv1D(
        filters=64,
        kernel_size=5,
        activation="relu",
        input_shape=(86, n_features)
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),
    Dropout(0.3),

    Conv1D(
        filters=32,
        kernel_size=3,
        activation="relu"
    ),
    BatchNormalization(),
    MaxPooling1D(pool_size=2),
    Dropout(0.3),

    Flatten(),

    Dense(64, activation="relu"),
    Dropout(0.3),

    Dense(3, activation="softmax")
])

temporal_cnn.compile(
    optimizer=Adam(learning_rate=0.0005),
    loss="categorical_crossentropy",
    metrics=["accuracy"]
)

temporal_cnn.summary()
```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass a n `input\_shape`/`input\_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(activity_regularizer=activity_regularizer, **kwargs)

Model: "sequential_8"

Layer (type)	Output Shape	Param #
conv1d_10 (Conv1D)	(None, 82, 64)	2,944
batch_normalization_16 (BatchNormalization)	(None, 82, 64)	256
max_pooling1d_5 (MaxPooling1D)	(None, 41, 64)	0
dropout_24 (Dropout)	(None, 41, 64)	0
conv1d_11 (Conv1D)	(None, 39, 32)	6,176
batch_normalization_17 (BatchNormalization)	(None, 39, 32)	128
max_pooling1d_6 (MaxPooling1D)	(None, 19, 32)	0
dropout_25 (Dropout)	(None, 19, 32)	0
flatten_5 (Flatten)	(None, 608)	0
dense_22 (Dense)	(None, 64)	38,976
dropout_26 (Dropout)	(None, 64)	0
dense_23 (Dense)	(None, 3)	195

Total params: 48,675 (190.14 KB)
Trainable params: 48,483 (189.39 KB)

Non-trainable params: 192 (768.00 B)

```
In [40]: # =====  
# 8. ENTRENAMIENTO  
# =====  
  
early_stop = EarlyStopping(  
    monitor="val_loss",  
    patience=15,  
    restore_best_weights=True  
)  
  
history_temporal = temporal_cnn.fit(  
    X_train_time_scaled,  
    y_train_time,  
    validation_split=0.2,  
    epochs=80,  
    batch_size=64,  
    class_weight=class_weights_manual,  
    callbacks=[early_stop],  
    verbose=1  
)
```

Epoch 1/80
119/119 ————— 9s 31ms/step - accuracy: 0.5140 - loss: 1.7494 - val_accuracy: 0.5909 - val_loss: 0.8946

Epoch 2/80
119/119 ————— 2s 19ms/step - accuracy: 0.5613 - loss: 1.3730 - val_accuracy: 0.6134 - val_loss: 0.8470

Epoch 3/80
119/119 ————— 4s 28ms/step - accuracy: 0.5791 - loss: 1.2708 - val_accuracy: 0.6113 - val_loss: 0.8492

Epoch 4/80
119/119 ————— 9s 60ms/step - accuracy: 0.5845 - loss: 1.2458 - val_accuracy: 0.6098 - val_loss: 0.8512

Epoch 5/80
119/119 ————— 7s 56ms/step - accuracy: 0.6038 - loss: 1.2273 - val_accuracy: 0.6161 - val_loss: 0.8482

Epoch 6/80
119/119 ————— 4s 30ms/step - accuracy: 0.6225 - loss: 1.1950 - val_accuracy: 0.6355 - val_loss: 0.8381

Epoch 7/80
119/119 ————— 2s 20ms/step - accuracy: 0.6211 - loss: 1.1857 - val_accuracy: 0.6266 - val_loss: 0.8473

Epoch 8/80
119/119 ————— 2s 19ms/step - accuracy: 0.6180 - loss: 1.1827 - val_accuracy: 0.6345 - val_loss: 0.8283

Epoch 9/80
119/119 ————— 2s 20ms/step - accuracy: 0.6301 - loss: 1.1682 - val_accuracy: 0.6371 - val_loss: 0.8221

Epoch 10/80
119/119 ————— 3s 22ms/step - accuracy: 0.6238 - loss: 1.1671 - val_accuracy: 0.6324 - val_loss: 0.8230

Epoch 11/80
119/119 ————— 2s 19ms/step - accuracy: 0.6349 - loss: 1.1564 - val_accuracy: 0.6413 - val_loss: 0.8278

Epoch 12/80
119/119 ————— 2s 17ms/step - accuracy: 0.6365 - loss: 1.1420 - val_accuracy: 0.6439 - val_loss: 0.8142

Epoch 13/80
119/119 ————— 3s 23ms/step - accuracy: 0.6349 - loss: 1.1456 - val_accuracy: 0.6423 - val_loss: 0.8225

Epoch 14/80
119/119 ————— 6s 27ms/step - accuracy: 0.6359 - loss: 1.1395 - val_accuracy: 0.6318 - val_loss: 0.8188

Epoch 15/80
119/119 ————— 3s 22ms/step - accuracy: 0.6377 - loss: 1.1282 - val_accuracy: 0.6397 - val_loss: 0.8206

Epoch 16/80
119/119 ————— 2s 19ms/step - accuracy: 0.6423 - loss: 1.1258 - val_accuracy: 0.6481 - val_loss: 0.8248

Epoch 17/80
119/119 ————— 3s 21ms/step - accuracy: 0.6470 - loss: 1.1141 - val_accuracy: 0.6429 - val_loss: 0.8160

Epoch 18/80
119/119 ————— 3s 23ms/step - accuracy: 0.6479 - loss: 1.1198 - val_accuracy: 0.6429 - val_loss: 0.8133

Epoch 19/80
119/119 ————— 2s 18ms/step - accuracy: 0.6519 - loss: 1.0989 - val_accuracy: 0.6497 - val_loss: 0.8099

Epoch 20/80
119/119 ————— 2s 19ms/step - accuracy: 0.6452 - loss: 1.1104 - val_accuracy: 0.6402 - val_loss: 0.8199

Epoch 21/80
119/119 ————— 3s 19ms/step - accuracy: 0.6500 - loss: 1.0984 - val_accuracy: 0.6376 - val_loss: 0.8233

Epoch 22/80
119/119 ————— 2s 20ms/step - accuracy: 0.6612 - loss: 1.0798 - val_accuracy: 0.6266 - val_loss: 0.8315

Epoch 23/80
119/119 ————— 2s 20ms/step - accuracy: 0.6566 - loss: 1.0787 - val_accuracy: 0.6429 - val_loss: 0.8119

Epoch 24/80
119/119 ————— 2s 16ms/step - accuracy: 0.6603 - loss: 1.0816 - val_accuracy: 0.6345 - val_loss: 0.8269

Epoch 25/80
119/119 ————— 3s 16ms/step - accuracy: 0.6657 - loss: 1.0666 - val_accuracy: 0.6366 - val_loss: 0.8229

Epoch 26/80
119/119 ————— 3s 19ms/step - accuracy: 0.6653 - loss: 1.0622 - val_accuracy: 0.6402 - val_loss: 0.82

```

08
Epoch 27/80
119/119 ————— 3s 25ms/step - accuracy: 0.6604 - loss: 1.0629 - val_accuracy: 0.6402 - val_loss: 0.82
56
Epoch 28/80
119/119 ————— 5s 20ms/step - accuracy: 0.6674 - loss: 1.0497 - val_accuracy: 0.6303 - val_loss: 0.82
77
Epoch 29/80
119/119 ————— 2s 19ms/step - accuracy: 0.6739 - loss: 1.0379 - val_accuracy: 0.6366 - val_loss: 0.82
91
Epoch 30/80
119/119 ————— 2s 17ms/step - accuracy: 0.6762 - loss: 1.0350 - val_accuracy: 0.6397 - val_loss: 0.84
13
Epoch 31/80
119/119 ————— 2s 20ms/step - accuracy: 0.6889 - loss: 1.0145 - val_accuracy: 0.6387 - val_loss: 0.82
54
Epoch 32/80
119/119 ————— 2s 19ms/step - accuracy: 0.6784 - loss: 1.0208 - val_accuracy: 0.6366 - val_loss: 0.83
28
Epoch 33/80
119/119 ————— 2s 19ms/step - accuracy: 0.6833 - loss: 1.0165 - val_accuracy: 0.6439 - val_loss: 0.83
44
Epoch 34/80
119/119 ————— 2s 18ms/step - accuracy: 0.6764 - loss: 1.0105 - val_accuracy: 0.6381 - val_loss: 0.84
04

```

```

In [41]: # =====
# 9. EVALUACIÓN
# =====

y_pred_prob = temporal_cnn.predict(X_test_time_scaled)
y_pred = np.argmax(y_pred_prob, axis=1)
y_true = np.argmax(y_test_time, axis=1)

print(classification_report(
    y_true,
    y_pred,
    target_names=le_time.classes_
))

print(confusion_matrix(y_true, y_pred))

```

```

75/75 ————— 1s 5ms/step
              precision    recall  f1-score   support

   SEG_A      0.76         0.85         0.80         1281
   SEG_B      0.59         0.45         0.51          670
   SEG_C      0.40         0.41         0.41          429

 accuracy                   0.66         2380
 macro avg              0.58         0.57         0.57         2380
weighted avg              0.65         0.66         0.65         2380

```

```

[[1084  93 104]
 [ 209 301 160]
 [ 137 114 178]]

```